

Toulouse INP - ENSEEIHT
École Nationale Supérieure d'Électrotechnique, d'Électronique, d'Informatique,
d'Hydraulique et des Télécommunications

RAPPORT DE STAGE DE FIN D'ÉTUDES
effectué chez Actimage

16 mars 2026 - 18 septembre 2026, 6 mois

Ingénierie Logicielle Full-Stack et DevOps

Vianney HERVY - Ingénieur Logiciel

vianney.hervy@etu.toulouse-inp.fr

Toulouse INP - ENSEEIHT
2 rue Charles Camichel
31071 Toulouse

Actimage
5 avenue Franco-Russe
75007 Paris

Responsable académique
Marc PANTEL
Maître de conférence en Informatique à l'IRIT /
ENSEEIHT

Encadrant chez Actimage
David KOLIN
(ex-)Directeur de l'agence Paris-Arcueil

1. Remerciements

Avant tout, je tiens à remercier mes deux encadrants sur cette expérience enrichissante qu'a été ce stage de fin d'étude. David pour son accompagnement, ses conseils et son humour, Matthias pour sa patience, sa bienveillance et ses succulents repas de fin de semaine.

Mes remerciements s'étendent à l'ensemble de mes collègues de l'agence Actimage Paris-Arcueil et d'ailleurs. Par ordre alphabétique: Aurélie, ma co-stagiaire solidaire pour son amitié, sa politesse et sa conversation ; Marine pour sa jovialité, sa créativité et son attention aux détails ; Romain pour sa curiosité et nos discussions technophiles ; et enfin Thomas, pour sa spontanéité et sa précieuse expertise footballistique.

Mes remerciements vont bien sûr également à Madeleine, ma fiancée, qui y est sûrement pour quelque chose dans la réussite de ce stage.

Table des matières

1. Remerciements	2
2. Introduction	4
2.1. Présentation de l'entreprise	4
2.1.1. Réalisations	4
DSFR	4
Site officiel du Gouvernement	4
BDnf	4
Hol'Autisme	4
2.1.2. Pôle développement	5
2.1.3. Pôle R et D	5
2.2. Contexte du stage	5
2.3. Gestion de projet	5
2.4. Poste de travail et outillage de développement	5
2.4.1. Environnement d'exécution et philosophie de travail	5
2.4.2. L'éditeur de code: le choix de Neovim	5
2.4.3. Contributions aux logiciels libres et outillage sur mesure	5
2.4.4. Le terminal comme espace de travail unifié	6
2.4.5. Conteneurisation et exécution locale	6
2.4.6. Évolution du poste de travail	6
3. Projet Office national des combattants et des victimes de guerre ¹ (ONaCVG)	6
3.1. Introduction	6
3.1.1. Présentation du client et genèse du besoin	6
3.1.2. L'existant : un défi de taille et de structure de la donnée	7
3.1.3. Migration et regroupement familial	7
3.1.4. Refonte logicielle et application de gestion état civil militaire (ECM).	7
Modélisation et consultation (Base ECM)	7
Module d'inspection en mobilité	8
Administration et flux de validation	8
3.1.5. Contraintes d'interopérabilité	8
3.2. Interlocuteurs, équipe et gestion de projet	8
3.2.1. Migration et regroupement familial : le défi de la réconciliation des données	8
Architecture et flux de traitement	9
Stratégies de résolution des conflits	9
Conteneurisation et enjeux de performance	10
3.3. Application principale	10
4. PIAWEB : Une histoire de DevOps	10
4.1. Le bogue	10
4.2. Le châtaignier	11
4.3. Appareillage sur lest	11
5. Conclusion	11
Bibliographie	12
Références logicielles	12
6. Annexes	13

¹<https://www.onac-vg.fr>

2. Introduction

2.1. Présentation de l'entreprise

Actimage est une entreprise de services numériques (ESN) française créée en 1995 par Christophe Megel, l'actuel PDG. Le travail de l'entreprise est divisé en 3 pôles principaux communicants: le développement, le conseil et la recherche et développement (R et D). Mon stage s'est déroulé dans le pôle développement mais j'ai eu l'occasion d'interagir avec le pôle R et D sur certains sujets. Les effectifs d'Actimage sont répartis entre 8 agences dans 5 pays. Celles avec lesquelles j'ai le plus été en contact sont Paris, Arcueil, Colmar, Strasbourg, Metz et Luxembourg.

2.1.1. Réalisations

DSFR

L'expertise d'Actimage en développement brille tout particulièrement sur les marchés publics. L'entreprise a notamment participé à la conception du Système de Design de l'État Français² (DSFR). Le DSFR regroupe un ensemble de règles et de composants réutilisables pour les interfaces officielles des sites en .gouv.fr. Il permet à l'État d'offrir des services numériques simples, accessibles et reconnaissables. C'est notamment celui que vous retrouvez sur impots.gouv.fr, ants.gouv.fr et sante.gouv.fr.

Site officiel du Gouvernement

L'un des sites majeurs du gouvernement est info.gouv.fr. C'est aussi un des projets principaux d'Actimage qui en réalise le développement dorsal et une partie du développement frontal. Plusieurs développeurs travaillent à temps plein sur ce projet, dont même certains physiquement au Service d'information du Gouvernement³ (SIG).

BDnf

Actimage est également l'entreprise derrière BDnF⁴, un outil ludo-éducatif de création de bandes dessinées et de récits multimédias développé pour le compte de la Bibliothèque nationale de France⁵ (BnF). Destinée principalement au milieu scolaire, l'application se connecte à la bibliothèque numérique Gallica via un module d'import permettant aux utilisateurs d'intégrer directement des corpus d'images d'archives dans leurs projets.

Techniquement, l'application a été développée de manière multi-support avec le moteur Unity 3D. L'architecture logicielle repose sur un noyau commun partagé entre les environnements de bureau (macOS, Windows) et tablettes (Android, iOS), tout en implémentant des fonctionnalités spécifiques adaptées aux contraintes des versions mobiles. Afin d'accélérer le développement des nouvelles fonctionnalités et de garantir la cohérence des interfaces sur toutes ces plateformes, l'équipe s'est appuyée sur la mise en place d'un système de design strict. Enfin, la conception a fait l'objet de tests d'utilisabilité itératifs menés directement auprès d'élèves pour valider l'ergonomie.

Hol'Autisme

Actimage s'inscrit fortement dans l'innovation et la R et D avec des projets à fort impact sociétal à l'image de Hol'Autisme⁶. Ce projet novateur propose le premier catalogue d'applications en réalité mixte destiné à aider les enfants et adolescents atteints de troubles du spectre autistique à développer leurs compétences sociales. Développée notamment avec le moteur Unity pour le casque HoloLens, la solution permet de simuler des situations du public ou du quotidien dans un environnement interactif et contrôlé. Le but est d'aider les

²<https://www.systeme-de-design.gouv.fr>

³<https://www.info.gouv.fr/organisation/service-d-information-du-gouvernement-sig>

⁴<https://bdnf.bnf.fr>

⁵<https://www.bnf.fr>

⁶<https://www.holautisme.com>

patients à appréhender les codes sociaux et à gagner progressivement en autonomie sans subir l'angoisse du monde réel.

L'expertise technologique du projet va bien au-delà de la simple réalité mixte : le système intègre un bracelet connecté permettant de mesurer le niveau d'anxiété de l'apprenant en temps réel, couplé à une plateforme web de contrôle et de suivi. Grâce à l'analyse de données et à des outils statistiques avancés, le personnel médico-éducatif peut analyser finement les sessions. La pertinence de ce dispositif global, dont la première preuve de concept s'intitule PopBalloons, a d'ailleurs été saluée par l'écosystème technologique, le projet étant lauréat des concours French IOT 2017 et Futur.e.s 2018.

2.1.2. Pôle développement

2.1.3. Pôle R et D

2.2. Contexte du stage

2.3. Gestion de projet

2.4. Poste de travail et outillage de développement

2.4.1. Environnement d'exécution et philosophie de travail

Au sein d'Actimage, le matériel mis à ma disposition était un poste opérant sous Windows 11. Afin de retrouver un environnement de développement familier, performant et conforme à la philosophie Unix qui guide mon flux de travail quotidien, j'ai fait le choix d'exploiter le sous-système Windows pour Linux [2]. J'y ai déployé une distribution Arch Linux. Cette approche m'a permis de répliquer quasi à l'identique l'environnement modulaire que j'utilise à titre personnel, tout en m'affranchissant des limitations inhérentes au système d'exploitation hôte pour le développement d'applications web.

2.4.2. L'éditeur de code: le choix de Neovim

Face à la complexité de l'écosystème PHP/Symfony, j'ai dans un premier temps évalué l'utilisation d'un environnement de développement intégré (IDE) classique, tel que PhpStorm [3]. Bien que cet outil propose des intégrations natives puissantes, sa lourdeur d'exécution et son manque de flexibilité (même pallié par l'extension IdeaVim [4]) ont constitué des freins à ma productivité. J'ai donc réintégré Neovim [5] comme éditeur principal.

Le langage PHP étant interprété et historiquement peu strict sur le typage, il ne bénéficie pas nativement des mêmes garanties à l'écriture qu'un langage compilé. Pour pallier cela et atteindre une rigueur de développement similaire à celle qu'offre TypeScript dans l'écosystème JavaScript, j'ai dû configurer un outillage robuste basé sur le Language Server Protocol (LSP) [6].

- J'ai principalement utilisé Phpactor [7] et Intelphense [8] pour l'autocomplétion et l'analyse statique.
- J'ai également expérimenté avec PHPantom [9] un serveur de langage récent développé en Rust [10], offrant des performances d'exécution nettement supérieures.
- L'intégration récente d'un serveur de langage dédié spécifiquement à Symfony [11] est venue parfaire cette configuration, me permettant d'avoir un retour intelligent sur les spécificités du cadre.

L'analyse de la qualité du code est restée isolée localement pour chaque projet, tout en étant exploitée par les serveurs de langage pour remonter les erreurs directement dans l'éditeur.

2.4.3. Contributions aux logiciels libres et outillage sur mesure

Constatant que certains outils de l'écosystème manquaient de maturité par rapport à d'autres langages, j'ai adopté une démarche proactive en contribuant à des projets à code source ouvert. J'ai notamment signalé et résolu des anomalies sur Twiggy [12] (un serveur de langage pour le moteur de gabarits Twig) et amélioré la grammaire Tree-sitter [13] de Twig, utilisée par Neovim pour l'analyse syntaxique. Cette implication m'a

également amené à échanger avec Fabien Potencier, créateur de Symfony, autour du développement de leur nouveau serveur de langage.

Pour optimiser la navigation dans les bases de code PHP, j'ai également développé des requêtes Tree-sitter personnalisées pour le greffon vim-matchup [14]. Celles-ci offrent une navigation syntaxique avancée en définissant des portées spécifiques pour les balises et les structures de contrôle du langage. Il devient ainsi possible de naviguer intelligemment entre les différentes clauses d'une condition ou d'une boucle, de parcourir aisément les blocs complexes (tels que switch, match ou try/catch), et de sauter instantanément de la signature d'une fonction à ses instructions de retour.

2.4.4. Le terminal comme espace de travail unifié

La reproduction de mon environnement s'arrêtant aux frontières du WSL et donc du terminal, j'ai optimisé ce dernier pour limiter au maximum l'usage de la souris et la friction liée aux changements de contexte. Le multiplexeur tmux [15] a été central dans cette démarche en me permettant de gérer de multiples invites de commande au sein d'une même fenêtre.

Afin de fluidifier mon flux de travail, j'ai enrichi cet espace de scripts et de raccourcis sur mesure. Ces personnalisations me permettent d'invoquer des interfaces interactives telles que lazygit [16] et lazydocker [17] sous forme de fenêtres superposées sans jamais quitter mon contexte d'édition, mais également d'interagir nativement au clavier avec des liens hypertextes ou d'accéder instantanément à l'interface web du dépôt Git du projet courant.

2.4.5. Conteneurisation et exécution locale

L'ensemble des projets, tels que PIAWEB, s'exécutaient au sein de conteneurs Docker [18]. Après une première phase d'utilisation de Docker Desktop pour Windows, j'ai rapidement constaté un manque de granularité et une interface graphique ralentissant mon flux de travail.

J'ai par conséquent migré vers une installation native du démon Docker exclusivement au sein de WSL. Ce choix m'a garanti un contrôle absolu sur les ressources et les conteneurs, pilotables intégralement en ligne de commande ou via l'interface terminale de lazydocker, en parfaite adéquation avec le reste de mon outillage.

2.4.6. Évolution du poste de travail

Travaillant pour la première fois dans un contexte extra-scolaire et extra-personnel pendant aussi longtemps, je trouve encore chaque semaine des points de friction, des tâches répétitives à optimiser ou automatiser. Mon poste de travail est en évolution constante, s'adaptant aux besoins de mon environnement de travail et de mes projets.

3. Projet ONaCVG

Le projet principal sur lequel j'ai eu l'occasion de travailler tout au long des 6 mois de stage est une application web à destination de l'ONaCVG. **TODO: étoffer**

3.1. Introduction

3.1.1. Présentation du client et genèse du besoin

L'ONaCVG est un établissement public administratif français placé sous la tutelle du ministère des Armées et des Anciens Combattants [1]. Fondé en 1916, cet organisme a pour vocation d'assurer des missions de reconnaissance, de réparation, de solidarité et de mémoire envers les combattants, les anciens combattants et les victimes de guerre [1]. L'Office opère au bénéfice d'environ 1,81 million de ressortissants (selon des estimations de 2023) à travers un réseau de services de proximité et s'impose comme l'opérateur majeur de la politique mémorielle du ministère des Armées [1].

Parmi ses prérogatives, l'ONaCVG exerce une compétence juridique spécifique en matière de sépultures militaires, un domaine encadré par le Code des pensions militaires d'invalidité et des victimes de guerre (CPMIVG) [1]. L'institution est explicitement chargée de la mise en œuvre de l'entretien, de la rénovation et de la valorisation des sépultures de guerre [1]. En effet, la loi pose le principe d'une sépulture perpétuelle pour les militaires déclarés « Mort pour la France », qu'ils reposent au sein de nécropoles nationales ou de carrés militaires communaux, et dont l'entretien incombe à l'État [1].

C'est dans le cadre de la gestion de ce vaste patrimoine funéraire et historique, et afin de moderniser ses outils numériques, que l'institution a lancé un appel d'offres visant à concevoir une nouvelle application centralisée de gestion des sépultures. Ce marché a été remporté par Actimage en **TODO: insérer date**. Le périmètre du contrat couvre la conception, le développement, la tierce maintenance applicative (TMA) ainsi que l'hébergement du futur service. L'application logicielle développée s'adresse exclusivement à un usage interne, ses utilisateurs finaux étant principalement les chefs de secteur de l'ONaCVG œuvrant sur le terrain et les administrateurs du pôle ECM.

3.1.2. L'existant : un défi de taille et de structure de la donnée

Le principal enjeu de ce projet réside dans l'héritage technique des données. La base de données existante (sous format MS Access [19]) recense plus de 800 000 sépultures, dont certaines remontent aux guerres napoléoniennes. Cette base historique compile une multitude d'informations : état civil militaire, nom et type du site, mentions honorifiques (telles que « Mort pour la France »), nationalité, informations de recrutement, unité militaire, ou encore causes du décès.

Cependant, le départ de la personne en charge de sa maintenance a entraîné une dégradation de l'intégrité des données, transformant la base en un document tabulaire peu rigoureux. Pour pallier ce manque d'outil centralisé, plusieurs chefs de secteur avaient dupliqué la « base mère » pour maintenir leurs données localement. Cette pratique a conduit à l'émergence de multiples « bases filles » désynchronisées, comportant des identifiants conflictuels, des doublons et des incohérences.

3.1.3. Migration et regroupement familial

La première mission de mon stage, qui s'est étendue sur un mois, a consisté à développer un outil de migration indépendant. Son objectif était de regrouper les bases filles avec la base mère en détectant les conflits et en proposant des stratégies de résolution : historisation des entrées conflictuelles ou conservation des deux versions via une renumérotation intelligente. Ce premier projet, qui fera l'objet d'une section détaillée ultérieurement, a constitué une excellente porte d'entrée pour m'approprier l'environnement technique de l'entreprise (PHP, Symfony, Doctrine, PostgreSQL, Docker) et les données de l'ONaCVG.

3.1.4. Refonte logicielle et application de gestion ECM.

Une fois les données fusionnées (toujours sous un format tabulaire plat d'environ quarante colonnes), la mission principale de mon stage a pu débuter : le développement de l'application de gestion complète, structurée autour de trois grands axes fonctionnels.

Modélisation et consultation (Base ECM)

Afin d'éviter la duplication et de garantir l'intégrité future des données, une refonte complète du modèle de données a été nécessaire. Nous sommes passés d'un format plat hérité du CSV à une architecture relationnelle stricte (création d'entités distinctes pour les pays, départements, communes, grades, unités, bureaux de recrutement, etc.). Une part majeure de mon travail a été consacrée à l'élaboration de la commande d'importation, capable de transformer des données libres et peu rigoureuses en entités standardisées. Sur cette base saine, un module de consultation a été développé, offrant des interfaces de recherche avancée avec de multiples filtres pour explorer les données des soldats et des sites.

L'ONaCVG ayant la charge de sépultures à perpétuité, les chefs de secteur doivent inspecter leurs sites (parfois plus de 300 par secteur) au moins une fois par an. J'ai participé au développement d'une interface optimisée pour tablettes permettant la saisie d'inspections sur le terrain. L'agent peut y corriger les informations de la base et évaluer l'état des infrastructures (sol, barrières, stèles, plaques). Ces relevés alimentent ensuite un algorithme de calcul estimant les coûts de restauration pour les tombes et sites concernés.

Administration et flux de validation

Le troisième volet de l'application concerne les administrateurs ECM. Pour garantir la qualité de la base de données sur le long terme, un flux de travail (workflow) a été mis en place. Bien que certaines actions soient libres, la modification de champs sensibles par un chef de secteur nécessite l'approbation d'un administrateur. Ce processus est accompagné d'un système de notifications intra-application et de courriels automatisés.

3.1.5. Contraintes d'interopérabilité

Enfin, le système devait respecter une contrainte forte d'interopérabilité avec les services de l'État. Les données n'étant pas strictement confidentielles, elles sont rendues accessibles au grand public via le portail gouvernemental Mémoire des Hommes⁷. L'application développée intègre donc une fonctionnalité d'export mensuel générant un format de fichier très spécifique, garantissant l'alimentation continue et conforme de ce portail national.

3.2. Interlocuteurs, équipe et gestion de projet

Dans le cadre de ce projet, nos interlocuteurs de l'ONaCVG étaient Audrey Paolasini, cheffe du département des achats, Emmanuelle PORTUGAL, archiviste, **TODO: le reste**.

Dans la réalisation de ce projet, j'étais accompagné de Matthias en tant que chef de projet, Marine responsable de l'UI/UX, Aurélie en assistance chefferie de projet, rédaction de spécifications et élagement en développement **TODO: demander son rôle exact**, Brice en tant que *lead developer*, et Amine et moi-même en tant que développeurs.

Pour chaque fonctionnalité majeure de l'application ont lieu des ateliers entre nos interlocuteurs et Matthias, Marine et Brice. Ces ateliers précisent le cahier des charges initial de l'appel d'offre. En découlent des maquettes Figma [20] que Marine réalise et puis des spécifications fonctionnelles détaillées (SFD) basées sur les maquettes et le cahier des charges. Ensuite, Brice divise la fonctionnalité en tickets et les assigne à Amine ou moi en fonction de nos capacités et disponibilités.

3.2.1. Migration et regroupement familial : le défi de la réconciliation des données

Cette phase du projet a été particulièrement formatrice, marquant ma première immersion dans l'écosystème PHP, le cadriciel Symfony et l'ORM Doctrine. Le défi à relever consistait à consolider une « base mère » et de multiples « bases filles » (fournies sous forme de fichiers CSV). Au sein de ces bases, chaque sépulture est théoriquement identifiée par un entier unique : la colonne `sdr_num` (numéro de saisie des registres). Cependant, suite à la scission des bases et à l'ajout décentralisé de nouvelles entrées par les chefs de secteur, de nombreuses collisions d'identifiants sont apparues.

Une simple fusion automatisée était inenvisageable en raison de la nature ambiguë de ces collisions. Si deux lignes strictement identiques peuvent être dédoublonnées sans risque, le cas de lignes partageant le même `sdr_num` mais présentant des divergences s'avère complexe. Il peut s'agir d'une véritable collision (deux soldats distincts ayant reçu le même identifiant de manière isolée) ou d'une mise à jour légitime (un même soldat dont les informations ont été enrichies dans une base fille, par exemple avec l'ajout d'un surnom). L'absence de règle mathématique pour trancher ces cas a imposé le développement d'un outil de migration

⁷<https://memoiredeshommes.defense.gouv.fr>

interactif. Cet outil agit comme une preuve de concept (POC) destinée à détecter les conflits et à déléguer la stratégie de résolution à l'utilisateur.

Architecture et flux de traitement

Pour répondre à ce besoin, j'ai conçu une architecture transactionnelle asynchrone reposant sur quatre tables principales au sein d'une base de données PostgreSQL : TMP, MERE, CONFLICTS et HIST. Le flux de traitement s'articule autour des étapes suivantes :

- **Acquisition et traitement asynchrone (Étapes 1 et 2) :** L'utilisateur téléverse un fichier CSV sur le serveur (SERV) via une route d'API dédiée (/csv-upload). Ce téléversement déclenche un processus en arrière-plan, géré par le composant Symfony Messenger (messenger-worker), qui se charge d'insérer l'intégralité des données dans la table temporaire TMP.
- **Répartition automatique (Étape 3) :** Un algorithme compare ensuite les entrées de TMP avec celles de la base MERE. Les lignes sans conflit d'identifiant y sont intégrées directement. En revanche, les lignes soulevant une collision sur le sdr_num sont isolées dans la table CONFLICTS. La table TMP est ensuite purgée de l'entrée fille.
- **Interface de résolution manuelle :** L'application propose des interfaces web (développées avec le moteur de template Twig et le cadriciel Tailwind CSS) permettant de lister de manière paginée les conflits détectés (route /conflicts) et d'en afficher les écarts en exergue (route /conflicts/{sdrNum}).

Stratégies de résolution des conflits

Face à une collision signalée dans l'interface, l'utilisateur dispose de trois stratégies de résolution (opérées via l'API /resolve/{origDb}/{sdrNum}) :

1. **Insertion (Action 4.1) :** Si les deux entrées représentent des soldats différents (vraie collision), l'entrée fille est insérée dans la base MERE et supprimée de CONFLICTS.
2. **Écrasement (Action 4.2) :** Si l'entrée fille est une version enrichie et valide de l'entrée mère, l'ancienne entrée mère est archivée dans la table d'historisation HIST. L'entrée fille vient ensuite la remplacer dans MERE via une opération de mise à jour (UPSERT), puis est supprimée de CONFLICTS.
3. **Suppression (Action 4.3) :** Si l'entrée fille est jugée non pertinente, elle est retirée de la table CONFLICTS et sauvegardée dans HIST afin de garder la donnée accessible en cas de besoin.

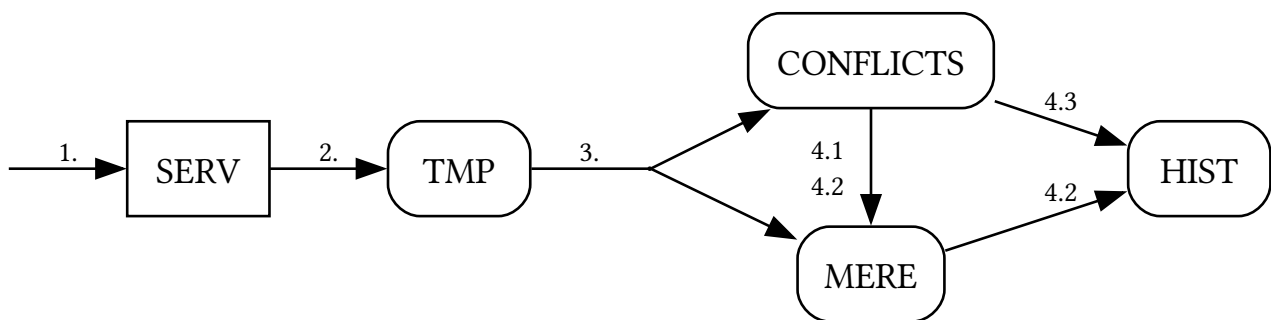
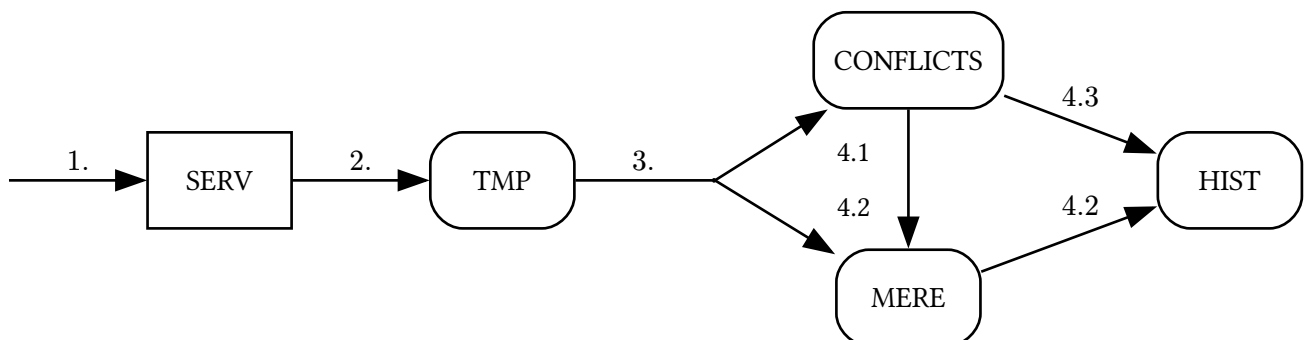
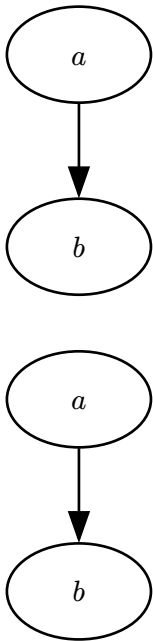


Fig. 1. – Flux de traitement de migration



digraph { a -> b }



Conteneurisation et enjeux de performance

L'ensemble de ce système a été pensé sous forme de micro-services conteneurisés avec Docker. L'environnement local orchestre la base de données (db), l'application web principale (app), le travailleur asynchrone (messenger-worker), la compilation à chaud des styles (tailwind) et le serveur web (nginx).

Cette phase initiale d'ingestion de données brutes a mis en évidence un point critique de performance : la traduction littérale des colonnes CSV (format plat) en colonnes PostgreSQL n'était pas viable à grande échelle. Tester l'outil sur de gros volumes de données (plus de 850 000 entrées générant parfois 80 000 conflits) a prouvé la nécessité absolue de scinder et de normaliser la base de données (création d'entités avec clés primaires et jointures) pour la future application principale, sous peine de subir des délais de latence rédhibitoires (timeouts) lors de la consultation.

3.3. Application principale

4. PIAWEB : Une histoire de DevOps

L'application PIAWEB⁸ dont Actimage réalise les développements et dirige les déploiements sur les serveurs clients est un projet de répertoire pour suivre les différentes actions du plan d'investissement France 2030 qui relèvent spécifiquement du Ministère de l'Enseignement Supérieur et de la Recherche (MESR).

Ce projet est actuellement en phase de TMA, peu de développements sont réalisés, majoritairement des corrections de bogues ou des évolutions mineures. La pile technologique repose sur du Spring et du Angular, une solution légèrement plus élaborée que celle proposée pour l'onacvg puisqu'elle requiert deux conteneurs serveur distincts pour le web frontal et dorsal.

4.1. Le bogue

Étant l'un des développeurs les moins coûteux, j'ai été assigné à la correction d'un bogue d'affichage ordinaire pour lequel plusieurs de mes collègues avaient déjà imputé du temps. La plongée laborieuse dans le code source d'un projet dont la teneur m'échappait encore et dont le cadrage frontal ne m'était pas familier m'a contraint à optimiser mon débogage afin d'identifier la source du problème sans avoir à explorer l'entièreté de l'application. Quelques échanges de tickets avec le client plus tard et j'arrivais à reproduire le comportement

⁸Contraction de Programme d'Investissements d'Avenir (PIA) et de Web.

anormal sur mon poste. Le problème venait d'un bête formulaire de recherche dont la pagination n'était pas rapportée à 1 lorsque l'utilisateur changeait les critères de recherche, permettant ainsi d'accéder à la troisième page pour une recherche ne remontant qu'une page de résultats par exemple.

Si l'implémentation du correctif ne nécessita que quelques minutes, la validation de la demande de fusion sur la branche de développement dev fut actée en une demi-heure. L'intervention aurait pu s'achever sur cette bonne note, mais l'équipe DevOps m'a confié la responsabilité de l'intégralité du cycle de livraison de cette version, incluant la montée sur les différents environnements et le déploiement final chez le client.

4.2. Le châtaignier

4.3. Appareillage sur lest

5. Conclusion

Bibliographie

- [1] Contributeurs de Wikipédia, « Office national des combattants et des victimes de guerre ». [En ligne]. Disponible sur: https://fr.wikipedia.org/wiki/Office_national_des_combattants_et_des_victimes_de_guerre

Références logicielles

- [2] Microsoft, « Windows Subsystem for Linux ». [En ligne]. Disponible sur: <https://wsl.dev/>
- [3] JetBrains, « PhpStorm ». [En ligne]. Disponible sur: <https://www.jetbrains.com/phpstorm>
- [4] JetBrains, « IdeaVim ». [En ligne]. Disponible sur: <https://lp.jetbrains.com/ideavim>
- [5] Contributeurs de Neovim, « Neovim ». [En ligne]. Disponible sur: <https://neovim.io/>
- [6] Microsoft, « LSP ». [En ligne]. Disponible sur: <https://microsoft.github.io/language-server-protocol>
- [7] Daniel Leech, « Phpactor ». [En ligne]. Disponible sur: <https://phpactor.readthedocs.io/>
- [8] Ben Mewburn, « Intelephense ». [En ligne]. Disponible sur: <https://intelephense.com/>
- [9] Anders Jenbo, « Phphantom ». [En ligne]. Disponible sur: https://phpantom-dev.github.io/phpantom_lsp
- [10] Rust Team, « Rust ». [En ligne]. Disponible sur: <https://rust-lang.org/>
- [11] Fabien Potencier, « Symfony Language Tools ». [En ligne]. Disponible sur: <https://github.com/symfony/language-tools>
- [12] Mikhail Gunin, « Twiggy ». [En ligne]. Disponible sur: <https://github.com/moetelo/twiggy>
- [13] Contributeurs de Tree-sitter, « Tree-sitter ». [En ligne]. Disponible sur: <https://tree-sitter.github.io/>
- [14] Andy Massimino, « Vim Matchup ». [En ligne]. Disponible sur: <https://github.com/andymass/vim-matchup>
- [15] Contributeurs de tmux, « tmux ». [En ligne]. Disponible sur: <https://github.com/tmux/tmux>
- [16] Jesse Duffield, « Lazygit ». [En ligne]. Disponible sur: <https://github.com/jesseduffield/lazygit>
- [17] Jesse Duffield, « Lazydocker ». [En ligne]. Disponible sur: <https://github.com/jesseduffield/lazydocker>
- [18] Docker Inc., « Docker ». [En ligne]. Disponible sur: <https://www.docker.com/>
- [19] Microsoft, « Microsoft Access ». [En ligne]. Disponible sur: <https://www.microsoft.com/fr-fr/microsoft-365/access>
- [20] Figma, « Figma ». [En ligne]. Disponible sur: <http://figma.com/>

6. Annexes